

How to be a Good Bean

Eduardo Pelegri-Llopart
Laurence P. G. Cable

September 1997



THE NETWORK IS THE COMPUTER™

JavaSoft

A Sun Microsystems, Inc. Business
901 San Antonio Road
Palo Alto, CA 94303 USA
415 960-1300 fax 415 969-9131

Copyright © 1997 by Sun Microsystems Inc.

901 San Antonio Road, Palo Alto, CA 94303.

All rights reserved.

RESTRICTED RIGHTS: Use, duplication or disclosure by the government is subject to the restrictions as set forth in subparagraph (c) (1) (ii) of the Rights in Technical Data and Computer Software Clause as DFARS 252.227- 7013 and FAR 52.227-19.

Sun, Sun Microsystems, the Sun logo, Java, JavaBeans, and JavaSoft, are trademarks or registered trade-marks of Sun Microsystems, Inc.

THIS PUBLICATION IS PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR USE, OR NON-INFRINGEMENT.

THIS PUBLICATION COULD INCLUDE TECHNICAL INACCURACIES OR TYPOGRAPHICAL ERRORS. CHANGES ARE PERIODICALLY ADDED TO THE INFORMATION HEREIN; THESE CHANGES WILL BE INCORPORATED IN NEW EDITIONS OF THE PUBLICATION. SUN MICROSYSTEMS, INC., MAY MAKE NEW IMPROVEMENTS AND/OR CHANGES IN THE PRODUCT(S) AND/OR THE PROGRAM(S) DESCRIBED IN THIS PUBLICATION AT ANY TIME.



Please
Recycle



Adobe PostScript



A JavaBeans component, or simply a Bean, is a reusable software component that can be manipulated visually in a builder tool. The JavaBeans 1.0 architecture specifies how a JavaBeans component exposes its features so tools can manipulate them to attain the needs of users.

The JavaBeans market has had a very strong start, both in tools and in components; in the near future it will keep growing in quantity and in quality. We want to promote commercial quality JavaBeans components and tools. This means components that can be used and reused by different users in different tools, can interoperate with other components from other vendors, and are robust and functionally complete.

The foundation to achieve these goals are the programming and packaging conventions as outlined in the JavaBeans specification (there is now a slightly updated 1.01 version). We provided additional guidelines for JavaBeans tools in the May 30, 1997 issue (#5) of The JavaBeans Advisor (see <http://java.sun.com/beans/Advisor.html>). Here we provide some additional guidelines for JavaBeans components developers to help ensure that the maximum level of component reuse is achieved.

Of course, these notes are not addressing the topic of effectively encapsulating your domain expertise into components. That expertise is your unique contribution, which will make your component successful.

Creating JavaBeans Components

Two rules implied by the JavaBeans architecture but that are sometimes missed include:

- the Bean class must provide zero-argument constructors so it can be created using **Beans.instantiate()**, and



- the Bean must support persistence, by implementing either `Serializable` or `Externalizable`.

And, of course, the usual considerations need to be taken for persistence, including the use of the **`transient`** keyword when using the default read and write methods.

Methods

Public methods describe the Bean's behavior. These methods are exposed by the builder tool. The methods are used by the builder tool or the user to construct connections between Beans.

Properties

For some important properties it may be useful to define events that are fired when the property changes in some specific way. A special example of this is a bound property, but other events may be fired as well.

We expect that the various tools vendors will do nice things for bounds properties so tool users can easily hook together properties on different JavaBeans components. Not all properties should be made available as a bound property but you should consider whether or not some useful properties of your Bean are to be made bound. For example, a button might be made bound at least for its font size and type, and background and foreground colors.

So if you have a property on one Bean where you want its value to be automatically propagated to a property on another Bean, then this is probably a good candidate for a bound property. For example, if you have a slider that is sliding between two values you may want to treat the current value as a bound property, because turning a slide movement on a slider Bean into a **`setFoo`** on a target Bean may be a very convenient user model.

Another example of useful bound property for a JavaBeans component is its internationalization locale: changing the locale is likely to be of interest to other components that are interacting with this one.

Constrained and Bound Properties

The JavaBeans architecture provides support for both constrained and bound properties. This support is orthogonal so that it is possible to have a property that is constrained but not bound. In practice, we strongly encourage Beans that export a property as constrained to also export it as a bound property; then listeners would register for both notifications. This allows a simple two-phase protocol for reacting to changes in one such property. When a property is about to change the listener is invoked through **VetoableChangeListener** and given an opportunity to veto the proposed change. Then when the change has actually happened, the (same) listener is invoked through **PropertyChangeListener** and the listener can react to the change.

Events

You can use BeanInfo to facilitate use of the Bean's events. The BeanInfo can also tailor access to the Bean's events by using the **hidden** and **expert** keywords. See below for more information on BeanInfo. Avoid misusing the event model--it is a notification scheme. Use default signatures for the event methods. This accelerates tool interaction.

BeanInfo

The overall goal of BeanInfo is to provide builder tools with enough information so that they can guide users in using the Bean. A large part of this is knowing how to subset the available choices for different users in different environments.

There are several aspects to creating good BeanInfo:

- For each feature (method, event, or property) decide whether it should be exposed at all to Bean users, and if so whether should be marked as hidden or expert. (Remember, not everything you inherit in implementing a Bean should necessarily be exposed.)

- Where applicable, provide a display name for the feature. Note that, for localization purposes, this name should be extracted from a resource bundle, so that different locales get useful localized feature names.
- Provide quality iconic representations. This makes the Bean easier to use and more recognizable within the builder tool.

Property Editors

Property editors allow for better manipulation of the Bean's properties. It allows you to customize your JavaBeans component on the type level of the properties. A property editor can be a Bean in itself and can be packaged together with the Bean it belongs to. You should deliver either a custom editor, or paintable and use the **getAsText()** method as last resort. A property editor can be linked to customizers via either BeanInfo or **PropertyEditorManager**.

Customizers

A Bean may need to be represented in a specific manner to users, or may need to be configured in a certain sequence. Builder tools have no advance knowledge of this. By providing a Customizer the JavaBeans component developer can address these situations. The Customizer has full access to the Bean and is normally packaged with the Bean. The Customizer can be a full-fledged JavaBeans component itself. A Customizer can set a private state of the Bean that the property sheet and methods and events do not have access to. Customizers do not override property editors. By using BeanInfo you can associate property editors with customizers.

One of the contributions of the Beans architecture is the improved separation between the run-time Bean behavior and the design-time behavior. The design-time behavior is exported through properties, their property editors, events, and through customizers.

You should consider what properties and events to expose, but these notes will not address this topic, except to point out that hidden properties and events can be used to expose them in a restricted manner.

A customizer can encapsulate deep expertise on the use of a component, and a customizer can be delivered independently of the Bean itself. Some JavaBeans components may even want to include different customizers, to address the needs of different customer' backgrounds, markets, sophistication, etc.

Packaging

JavaBeans components are delivered by means of a JAR file. A JAR that contains a Bean must include a Manifest declaring the Beans it contains, be it a class or a persistent representation of a prototype of the Bean.

In the absence of additional information, a Bean packaged in a given JAR file may require all the files in that JAR for its successful operation; thus a builder tool that uses JavaBeans components from a series of JAR files will need to include all files in these JARs in the delivered products (be them Beans, applets, or applications). The JavaBeans 1.01 specification provides a mechanism for refining this through the **Design-Time-Only** and **Depends-On** tags. Beans developers should consider using these headers for the benefit of tools. Refer to the 1.01 specification for details.

The JAR can also contain any other files--such as images, sounds, HTML help files--that the Bean relies on. The JAR can also contain different localized versions of this information. To support deployment of the Bean in a builder tool the JavaBeans component developer can insert Javadoc information into the JAR as well.

New Beans from Old Beans

JavaBeans components are intended to be reused. An end-user uses a Bean by instantiating it within a JavaBeans-compliant tool. But some developers may want to create new JavaBeans components by using your Bean. The JavaBeans architecture provides some mechanisms for this purpose that you should take into consideration:

- A new JavaBeans component can be created by instantiating a Bean, customizing it by manipulating its properties or through a Customizer, and then serializing the Bean.



-
- The functionality of a Bean can be exposed in new ways by providing a new Customizer targeted to a different audience.
 - A new Bean can be created by subclassing an existing Bean.